សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

Hotel Management System

បង្រៀនដោយសាស្ត្រាចារ្យ៖ Dan Vournpheng

ថ្នាក់ A4

សមាជិក

១. រ៉ុ ឌីរ៉ះ **២. ប៉ាត មូលសក្តា** **៣. នឹម រត្ត០ៈ** **៤. ទូ និចាន់** **៥. ឈឿន គុណា**

Hotel Management System

Desktop Application | C# Windows Forms

Layered Architecture · 5 Design Patterns · 12 Operational Forms

Objective

Functionalities
Forms 1–12

Design Patterns
5 Patterns

Conclusion

Demo

OBJECTIVE

Objective

What the Hotel Management System is capable of doing

System Purpose

The Hotel Management System is a comprehensive C# Windows Forms desktop application designed to digitize and streamline all daily hotel operations from a single, unified interface.

- Replaces manual paper-based hotel operations
- Reduces human error in booking & billing
- Real-time visibility into hotel room status
- Automates housekeeping task assignment
- Role-based access: Admin · Receptionist · Housekeeping

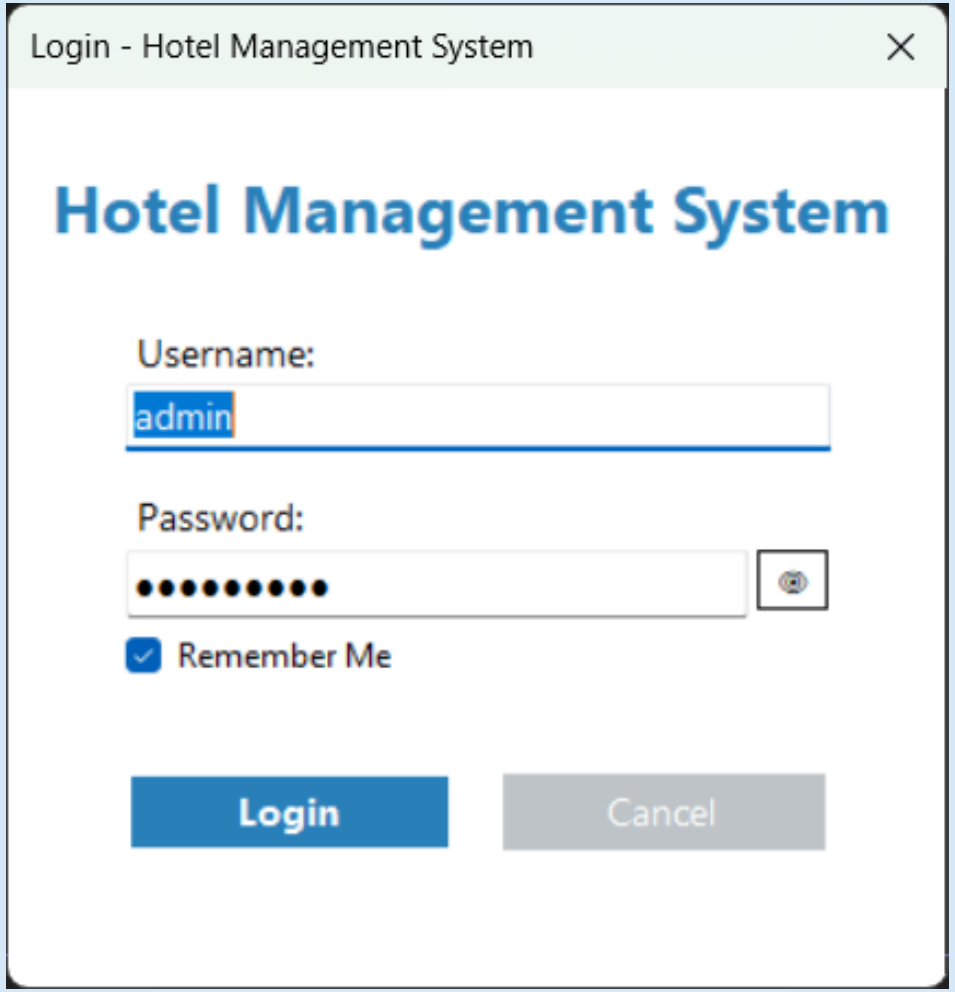
Core Capabilities

Authentication	Secure login, SHA-256 hashing, role-based access
Guest Management	Register, search, edit guests & view booking history
Room Management	4 room types, status tracking, visual dashboard
Booking	Create, confirm, check-in / check-out, cancel
Invoicing	Auto-generate invoice, 10% tax, discount support
Payment	Cash & Credit Card, styled receipt & print
Housekeeping	Auto-create tasks on checkout, smart staff assignment
Staff Management	CRUD accounts, role & password management
Reports	Daily operations dashboard & key metrics

FORM 1

Form 1 — Login Form

LoginForm.cs



Form Description

📌 **Purpose** Entry point — authenticates users before granting access

Key Features & Functions

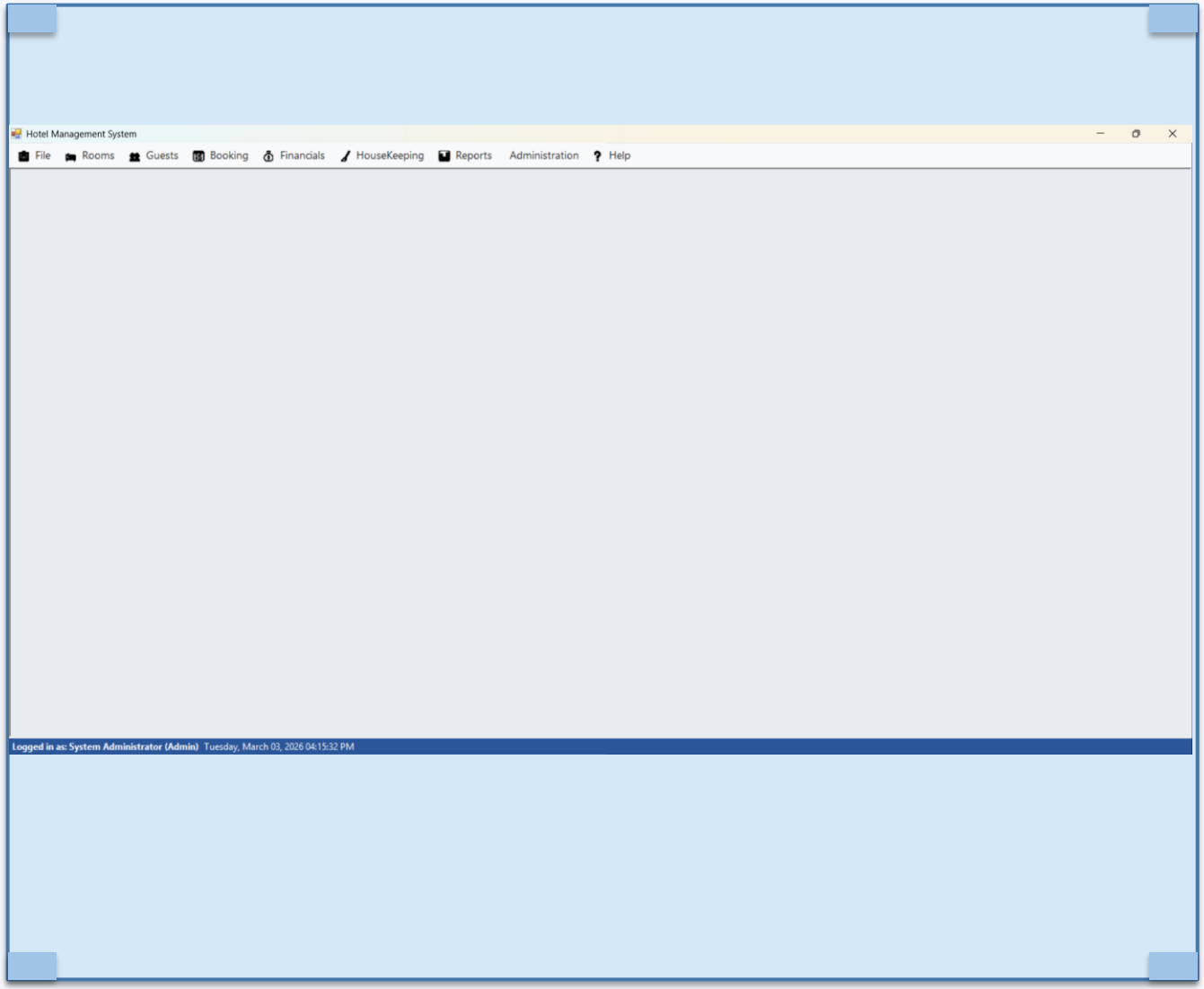
- Username & password fields with visibility toggle (🔒 /👁)
- SHA-256 + salt password hash verification
- 'Remember Me' stores credentials in app settings
- On success → stores user in SessionManager & opens MainForm
- On failure → shows error, clears password field
- Role-based access control: Admin / Receptionist / Housekeeping

💡 Uses Singleton: SessionManager.CurrentUser

FORM 2

Form 2 — Main Form (MDI Hub)

MainForm.cs



Form Description

📌 **Purpose** Central navigation container for all system modules

Key Features & Functions

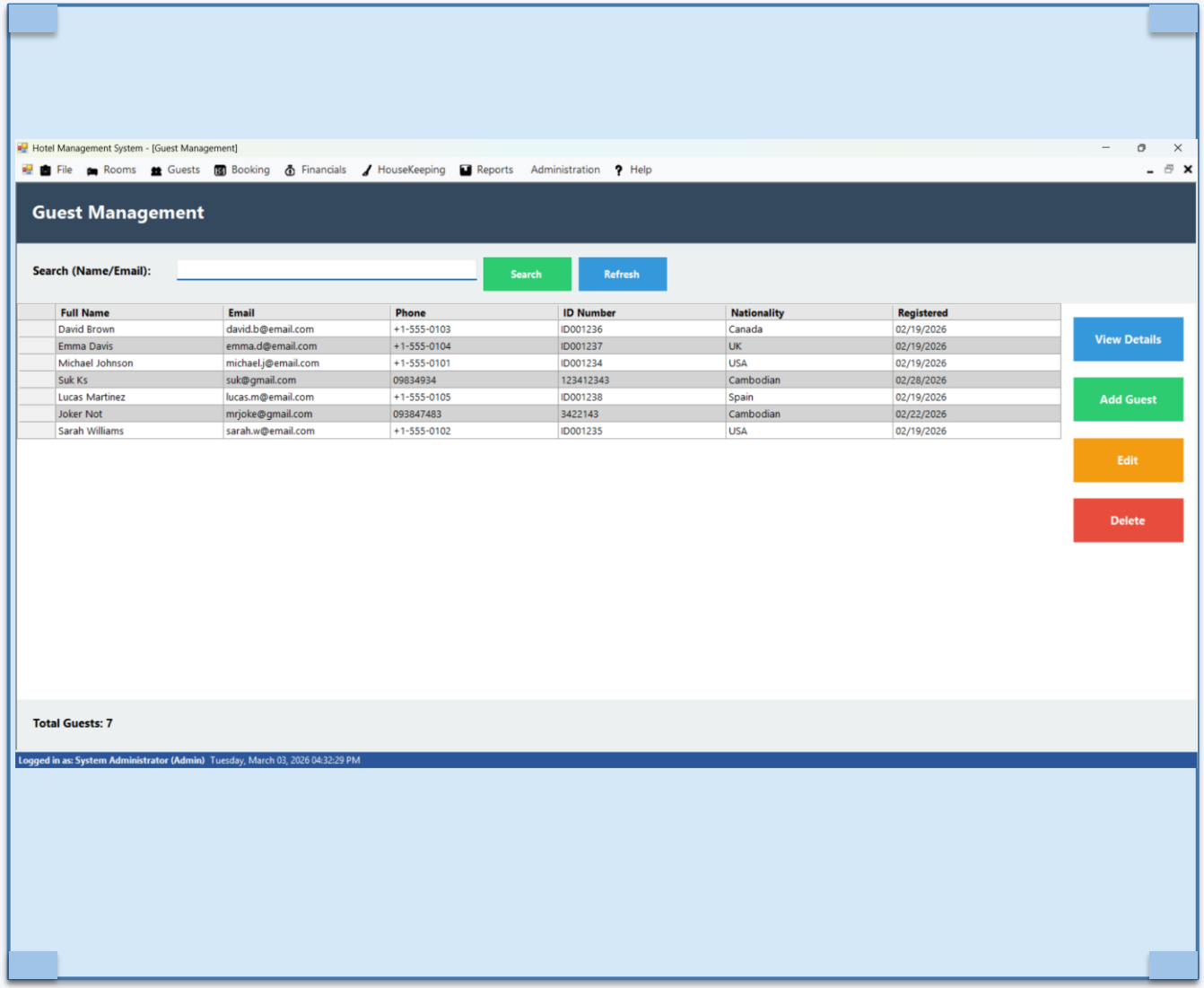
- MDI (Multiple Document Interface) parent window
- Menu bar: Booking, Guest, Room, Housekeeping, Invoices, Payments, Reports, Settings, Admin
- Status bar: current user, role, live clock (updates every second)
- Initializes Observer pattern on startup:
- RoomSubject.Instance.Attach(new HousekeepingObserver())
- Midnight-blue theme with Segoe UI typography

💡 Observer pattern initialized here — central event hub

FORM 3

Form 3 — Guest Management

GuestManagementForm.cs



Form Description

📌 **Purpose** Full CRUD interface for managing all hotel guests

Key Features & Functions

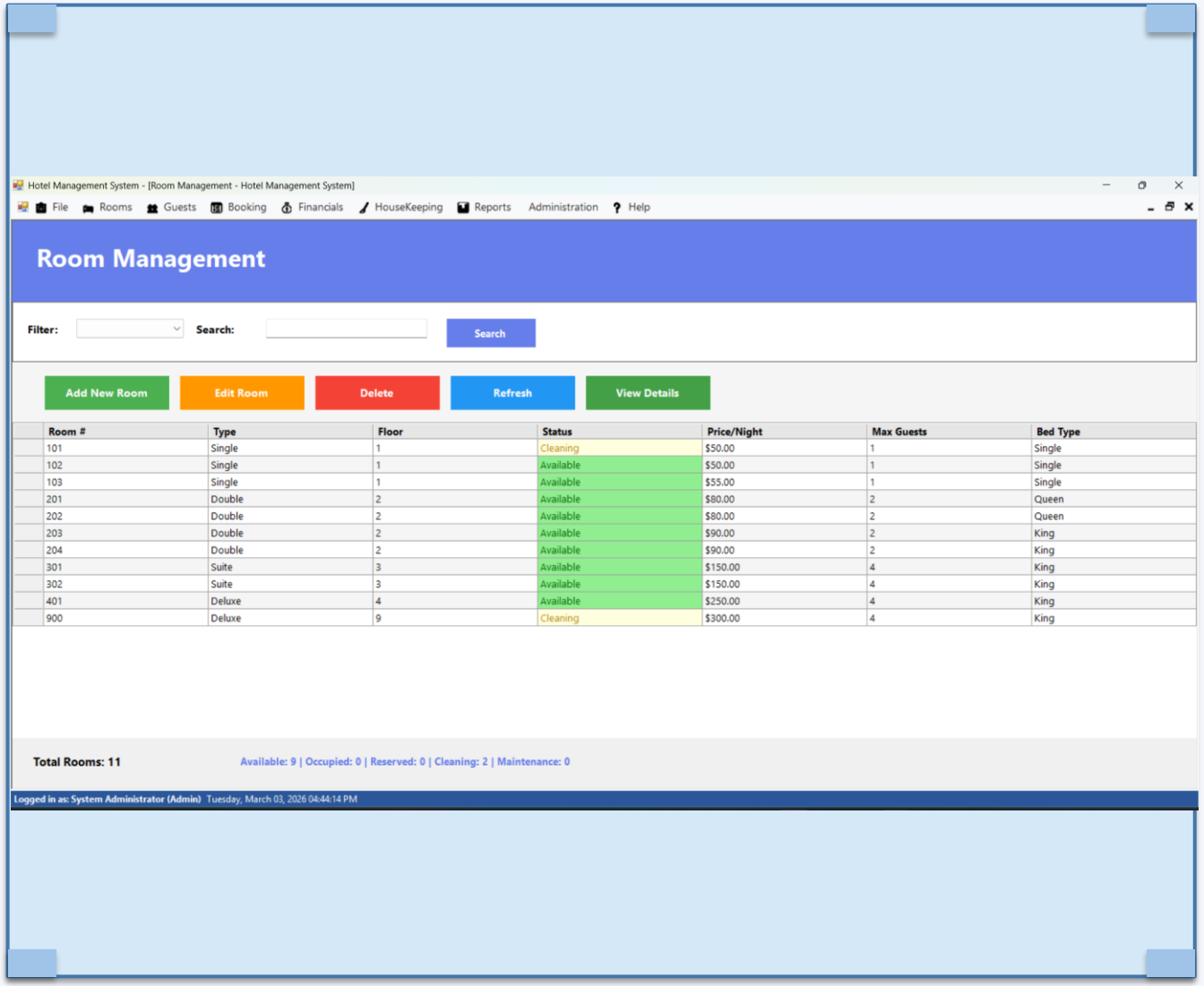
- DataGridView listing all active guests
- Columns: Name, Email, Phone, ID Number, Nationality
- Search by name, email, or phone number
- Add Guest → opens AddEditGuestDialog (insert new record)
- Edit Guest → opens dialog pre-populated with data
- Delete Guest → soft delete (sets IsActive = 0)
- View Details → opens GuestDetailsDialog (full profile)

💡 Soft delete preserves history — data is never hard-deleted

FORM 4

Form 4 — Room Management

RoomManagementForm.cs



Room #	Type	Floor	Status	Price/Night	Max Guests	Bed Type
101	Single	1	Cleaning	$50.00	1	Single
102	Single	1	Available	$50.00	1	Single
103	Single	1	Available	$55.00	1	Single
201	Double	2	Available	$80.00	2	Queen
202	Double	2	Available	$80.00	2	Queen
203	Double	2	Available	$90.00	2	King
204	Double	2	Available	$90.00	2	King
301	Suite	3	Available	$150.00	4	King
302	Suite	3	Available	$150.00	4	King
401	Deluxe	4	Available	$250.00	4	King
900	Deluxe	9	Cleaning	$300.00	4	King

Form Description

📌 **Purpose** Manage hotel rooms with a visual status dashboard

Key Features & Functions

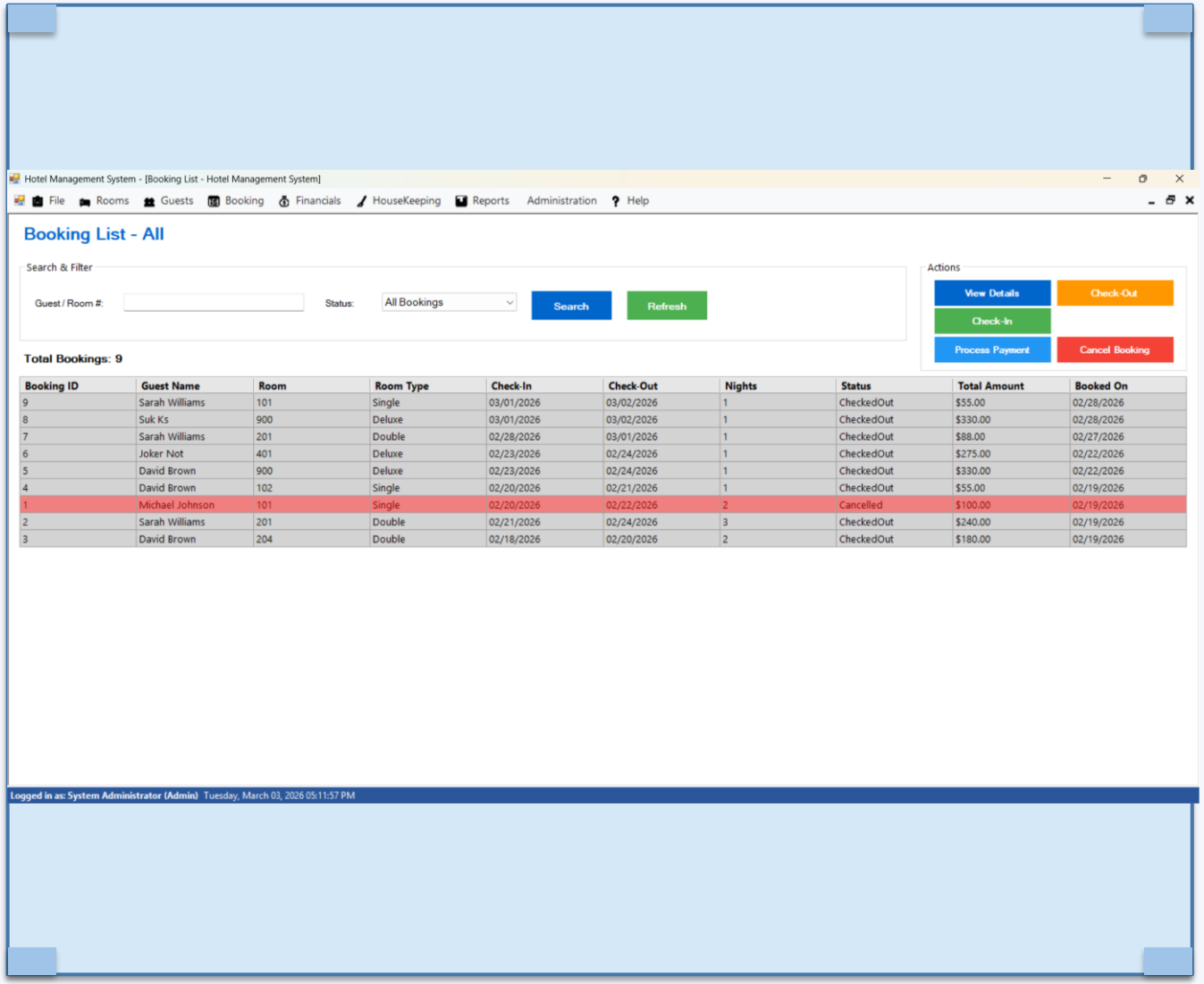
- Room list filterable by type and status
- Add Room → RoomFactory.CreateRoom() (Factory Pattern)
- 4 types: Single · Double · Suite · Deluxe
- Edit Room / Delete Room (soft delete → Maintenance)
- Room Status Dashboard — color-coded card overview
- Statuses: Available · Occupied · Reserved · Maintenance · Cleaning

💡 Factory Pattern: room creation logic centralized in RoomFactory

FORM 5

Form 5 — Booking List

BookingListForm.cs



Booking ID	Guest Name	Room	Room Type	Check-In	Check-Out	Nights	Status	Total Amount	Booked On
9	Sarah Williams	101	Single	03/01/2026	03/02/2026	1	CheckedOut	$55.00	02/28/2026
8	Suk Ks	900	Deluxe	03/01/2026	03/02/2026	1	CheckedOut	$330.00	02/28/2026
7	Sarah Williams	201	Double	02/28/2026	03/01/2026	1	CheckedOut	$88.00	02/27/2026
6	Joker Not	401	Deluxe	02/23/2026	02/24/2026	1	CheckedOut	$275.00	02/22/2026
5	David Brown	900	Deluxe	02/23/2026	02/24/2026	1	CheckedOut	$330.00	02/22/2026
4	David Brown	102	Single	02/20/2026	02/21/2026	1	CheckedOut	$55.00	02/19/2026
1	Michael Johnson	101	Single	02/20/2026	02/22/2026	2	Cancelled	$100.00	02/19/2026
2	Sarah Williams	201	Double	02/21/2026	02/24/2026	3	CheckedOut	$240.00	02/19/2026
3	David Brown	204	Double	02/18/2026	02/20/2026	2	CheckedOut	$180.00	02/19/2026

Form Description

📌 **Purpose** Central hub for all booking operations and status management

Key Features & Functions

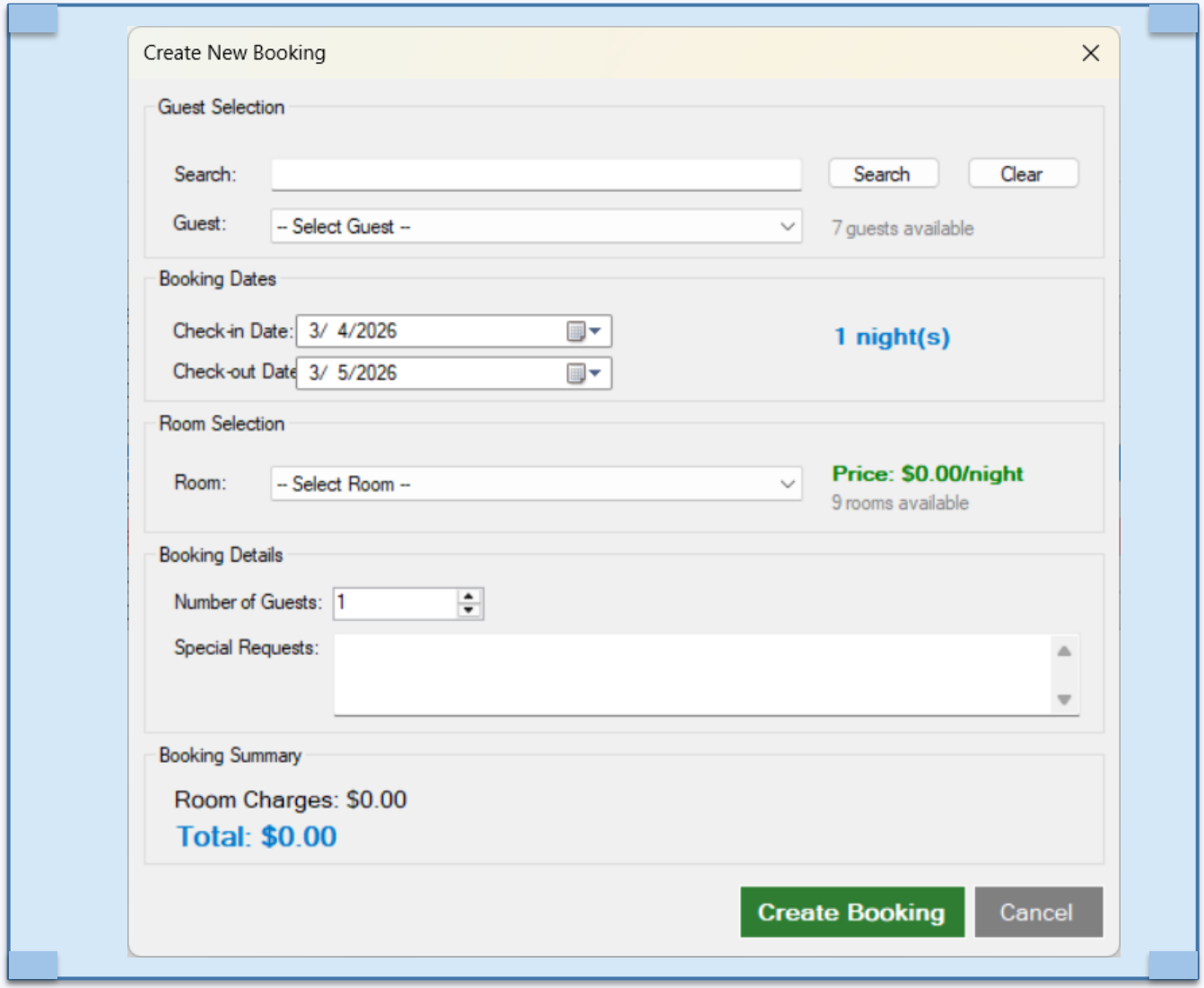
- DataGridView with color-coded booking status rows
- 🟡 Pending 🟢 Confirmed 🔵 CheckedIn ⚪ CheckedOut ⬜⬜ Cancelled
- Search & filter by guest name, room number, or status
- Check-In → validates status, sets room to Occupied
- Check-Out → sets room to Cleaning, auto-generates invoice, triggers Observer (auto-creates housekeeping task)
- Process Payment → opens PaymentForm
- Cancel Booking → prompts reason, frees the room

💡 Checkout triggers Observer → auto housekeeping task created

FORM 6

Form 6 — New Booking

NewBookingForm.cs



Form Description

Purpose Wizard-style form for creating new reservations

Key Features & Functions

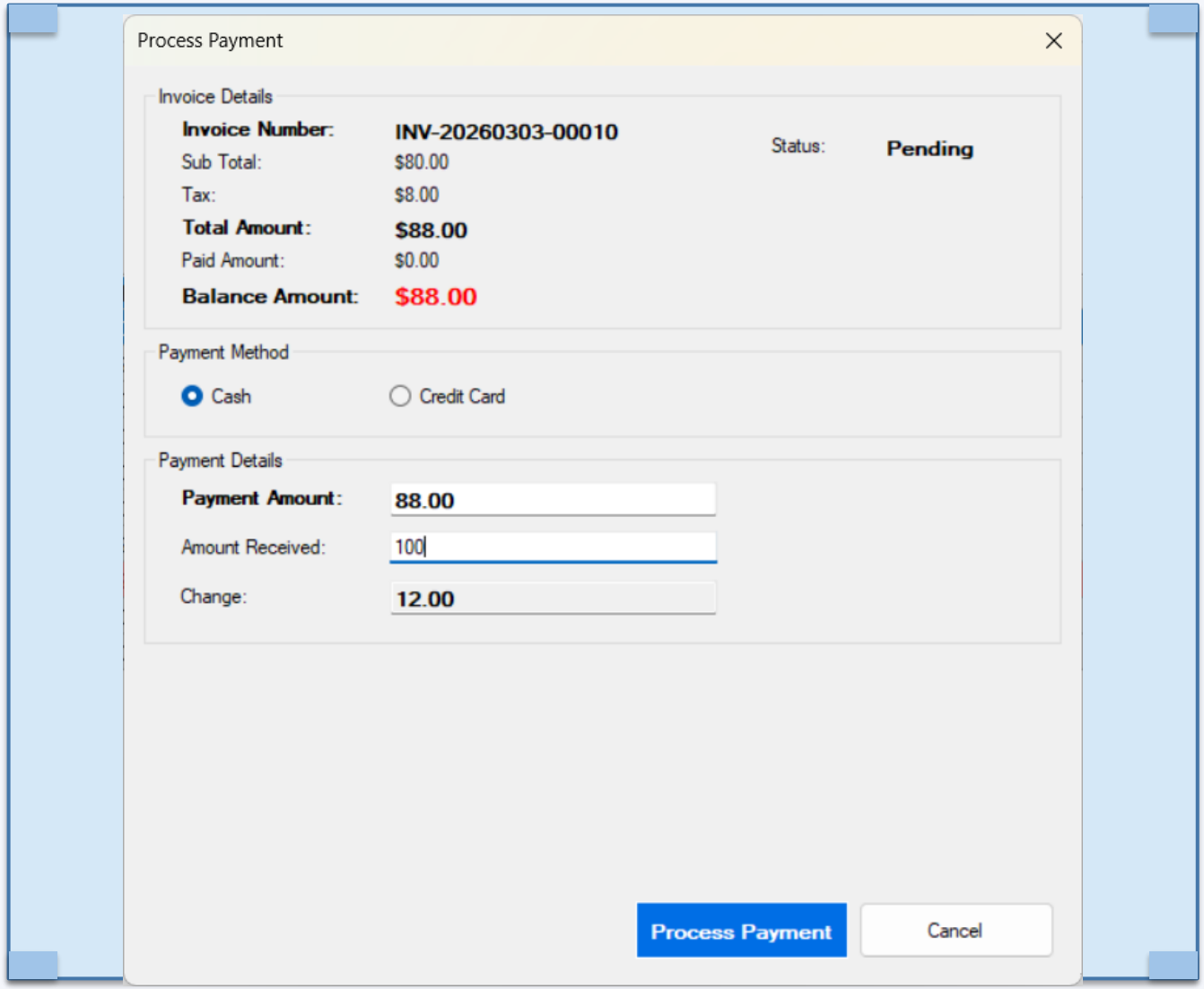
- Guest selection from registered guest list
- Room selection filtered by availability and date range
- Check-in / Check-out date pickers with conflict detection
- Auto charge calculation: BasePrice × Number of Nights
- Total with 10% tax via BookingFacade.CalculateTotalAmount()
- Facade Pattern orchestrates the full 8-step booking workflow
- Booking created with Status = Pending

Facade Pattern: CreateBooking() hides 8 internal steps

FORM 7

Form 7 — Payment Form

PaymentForm.cs



Form Description

📌 **Purpose** Processes guest payments using the Strategy design pattern

Key Features & Functions

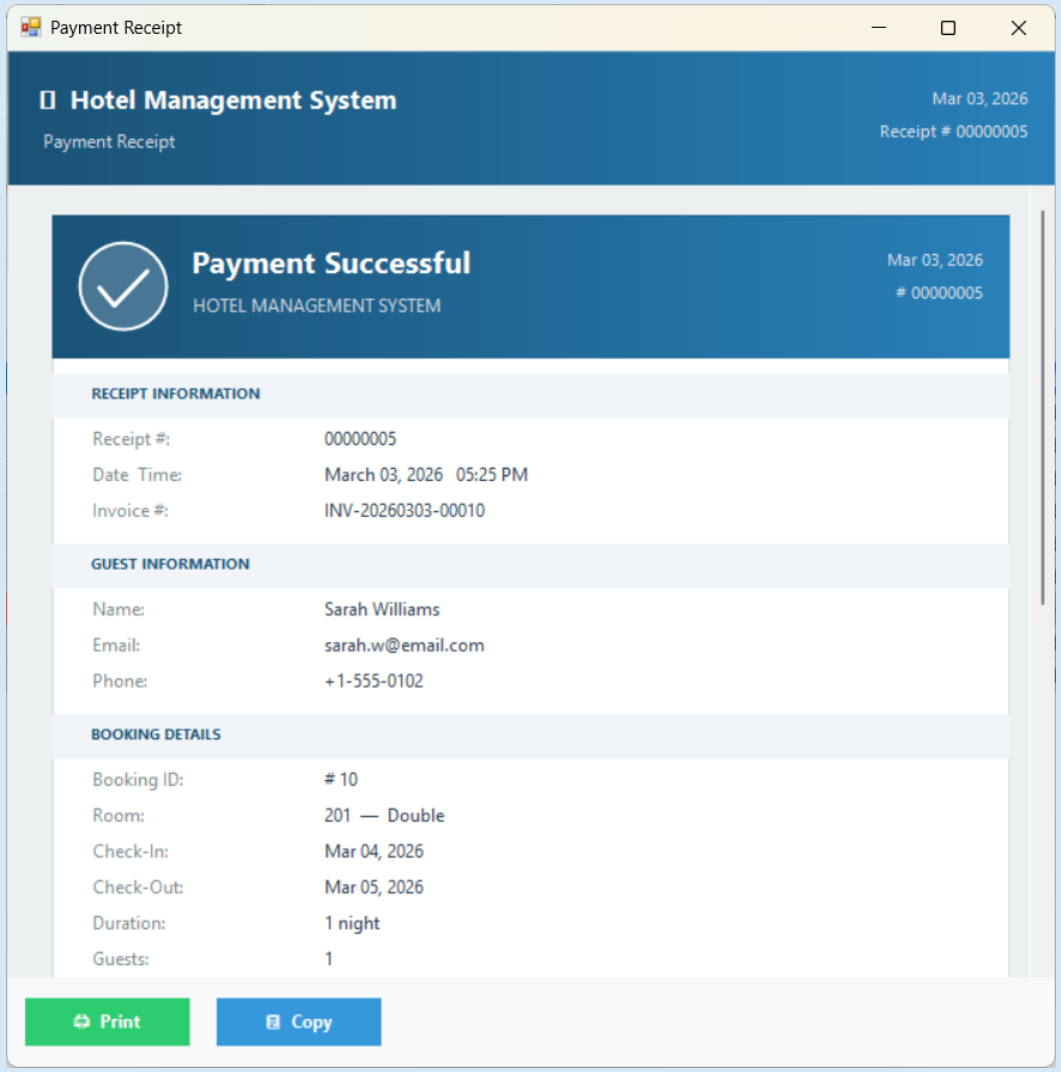
- Displays invoice: subtotal, tax, total, paid, balance due
- Cash mode: enter amount received → auto change calculation
- → Uses CashPaymentStrategy
- Credit Card mode: card number (Luhn validation), holder, expiry, CVV → simulated gateway (95% success rate)
- → Uses CreditCardPaymentStrategy
- Card number masked — only last 4 digits stored in DB
- On success → automatically opens Receipt Form

💡 Strategy Pattern: payment algorithm selected at runtime

FORM 8

Form 8 — Receipt Form

ReceiptForm.cs



Form Description

📌 **Purpose** Styled visual receipt displayed after successful payment

Key Features & Functions

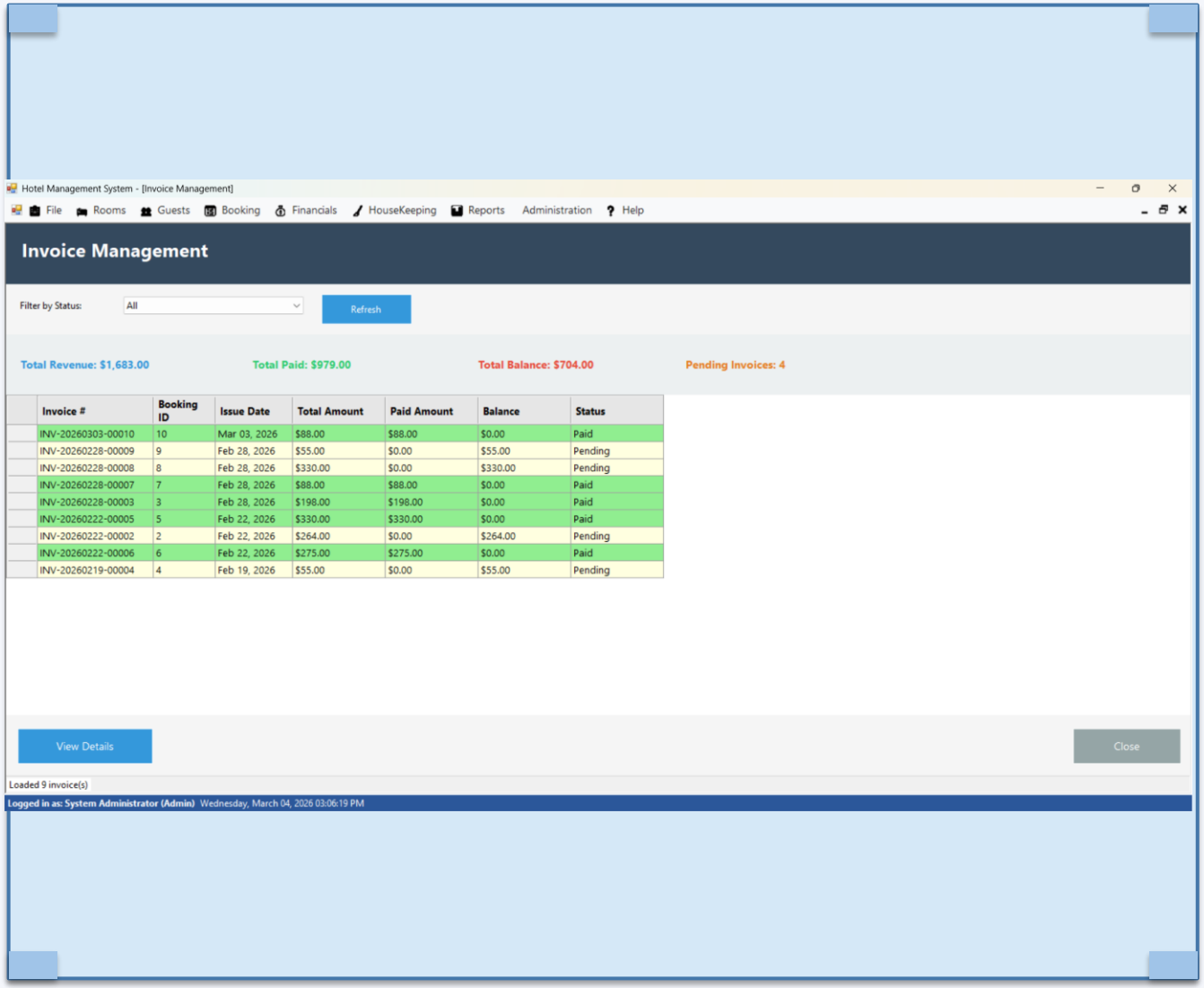
- Gradient blue header with checkmark + 'Payment Successful'
- Section cards: Receipt Info · Guest Info · Booking Details · Financial Breakdown · Payment Info
- Status badge: ✓ PAID IN FULL (green) · ◑ PARTIALLY PAID (orange) · ○ PENDING (red)
- Action buttons: Print · Export PDF · Copy to Clipboard · Close
- Custom GDI+ drawing with LinearGradientBrush for header

💡 Professional receipt — supports print & PDF export

FORM 9

Form 9 — Invoice Management

InvoiceListForm.cs / InvoiceForm.cs



Invoice #	Booking ID	Issue Date	Total Amount	Paid Amount	Balance	Status
INV-20260303-00010	10	Mar 03, 2026	$88.00	$88.00	$0.00	Paid
INV-20260228-00009	9	Feb 28, 2026	$55.00	$0.00	$55.00	Pending
INV-20260228-00008	8	Feb 28, 2026	$330.00	$0.00	$330.00	Pending
INV-20260228-00007	7	Feb 28, 2026	$88.00	$88.00	$0.00	Paid
INV-20260228-00003	3	Feb 28, 2026	$198.00	$198.00	$0.00	Paid
INV-20260222-00005	5	Feb 22, 2026	$330.00	$330.00	$0.00	Paid
INV-20260222-00002	2	Feb 22, 2026	$264.00	$0.00	$264.00	Pending
INV-20260222-00006	6	Feb 22, 2026	$275.00	$275.00	$0.00	Paid
INV-20260219-00004	4	Feb 19, 2026	$55.00	$0.00	$55.00	Pending

Form Description

📌 **Purpose** — View and manage all invoices generated from bookings

Key Features & Functions

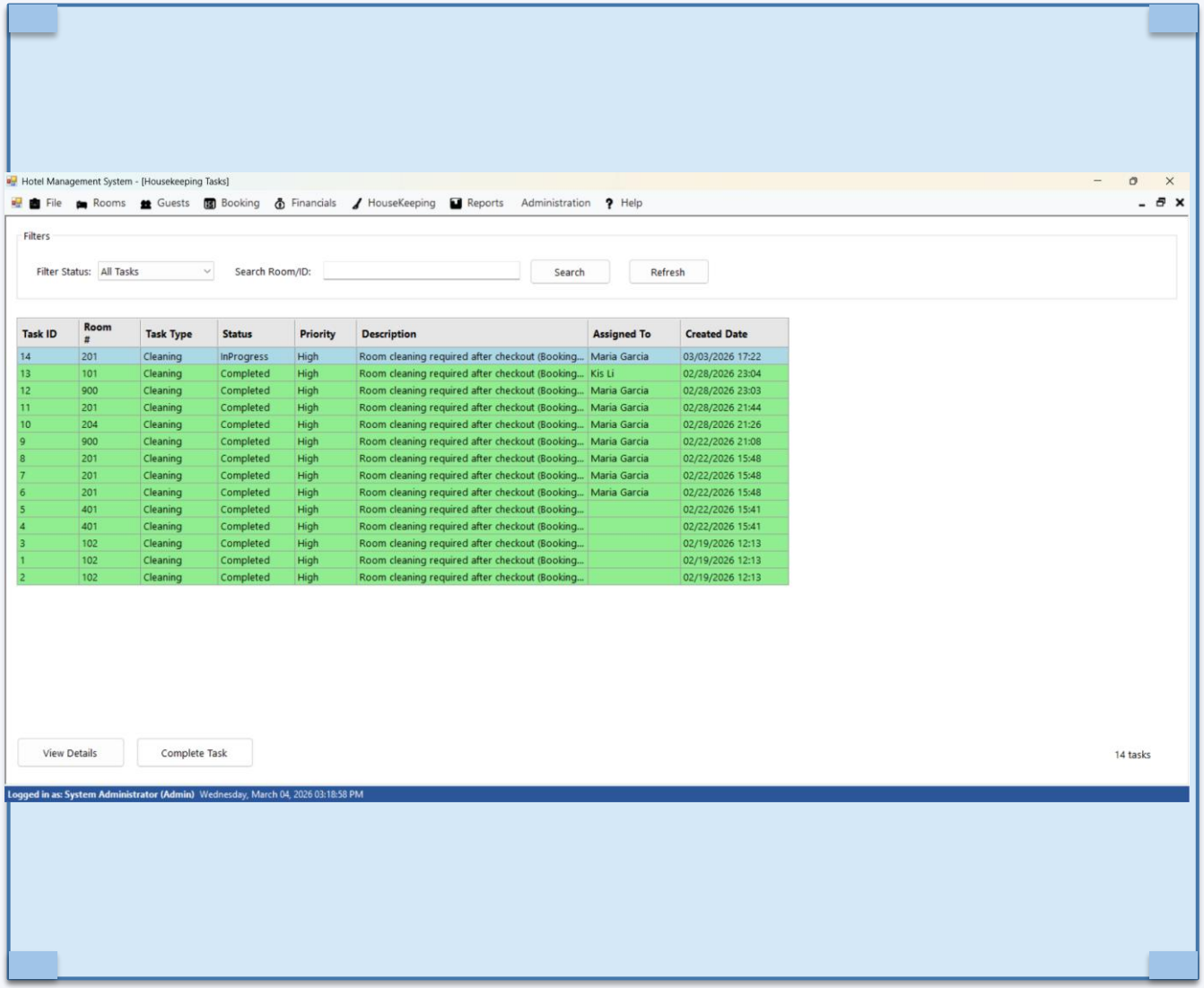
- List all invoices with status filtering
- Statuses: Pending · Paid · Partially Paid · Cancelled
- Invoice details: subtotal, 10% tax, discounts, total
- View full details via InvoiceDetailsDialog
- Track complete payment history per invoice
- Auto-generated at guest checkout — no manual entry needed

💡 Invoices are auto-generated on checkout — zero manual work

FORM 10

Form 10 — Housekeeping Tasks

HousekeepingTasksForm.cs



Task ID	Room #	Task Type	Status	Priority	Description	Assigned To	Created Date
14	201	Cleaning	InProgress	High	Room cleaning required after checkout (Booking...	Maria Garcia	03/03/2026 17:22
13	101	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Kis Li	02/28/2026 23:04
12	900	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/28/2026 23:03
11	201	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/28/2026 21:44
10	204	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/28/2026 21:26
9	900	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/22/2026 21:08
8	201	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/22/2026 15:48
7	201	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/22/2026 15:48
6	201	Cleaning	Completed	High	Room cleaning required after checkout (Booking...	Maria Garcia	02/22/2026 15:48
5	401	Cleaning	Completed	High	Room cleaning required after checkout (Booking...		02/22/2026 15:41
4	401	Cleaning	Completed	High	Room cleaning required after checkout (Booking...		02/22/2026 15:41
3	102	Cleaning	Completed	High	Room cleaning required after checkout (Booking...		02/19/2026 12:13
1	102	Cleaning	Completed	High	Room cleaning required after checkout (Booking...		02/19/2026 12:13
2	102	Cleaning	Completed	High	Room cleaning required after checkout (Booking...		02/19/2026 12:13

Form Description

📌 **Purpose** Manage all cleaning and maintenance tasks for rooms

Key Features & Functions

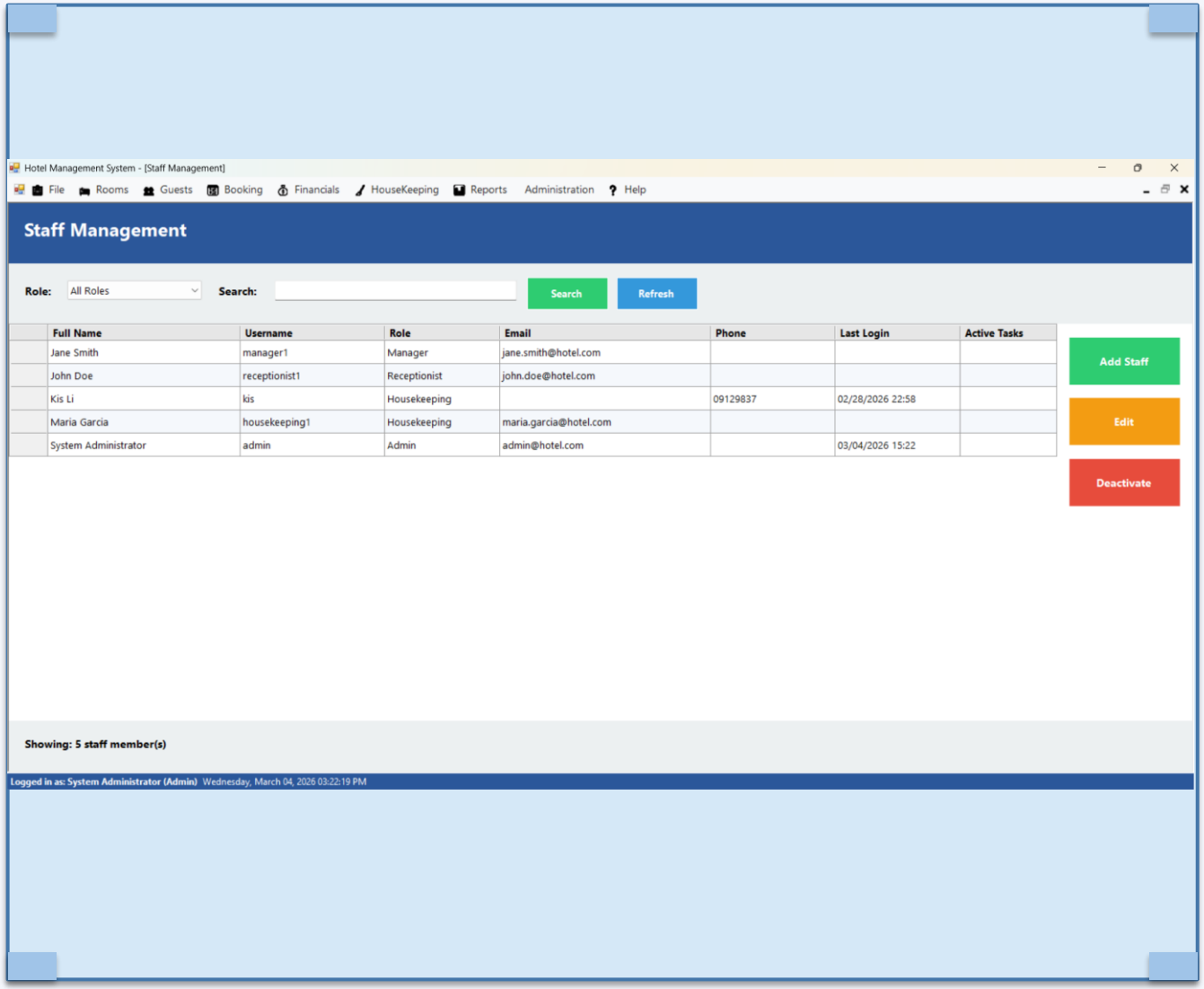
- Lists tasks with: status, priority, room number, assigned staff
- Tasks auto-created by Observer pattern when guest checks out
- Auto-assignment: least-busy housekeeping staff algorithm
- Manual assignment override also available
- Status workflow: Pending → In Progress → Completed
- Priority levels: High (post-checkout) / Normal (routine)

💡 Observer Pattern: task creation is fully automated on checkout

FORM 11

Form 11 — Staff Management

StaffManagementForm.cs



Form Description

📌 **Purpose** Admin-only interface for managing all system user accounts

Key Features & Functions

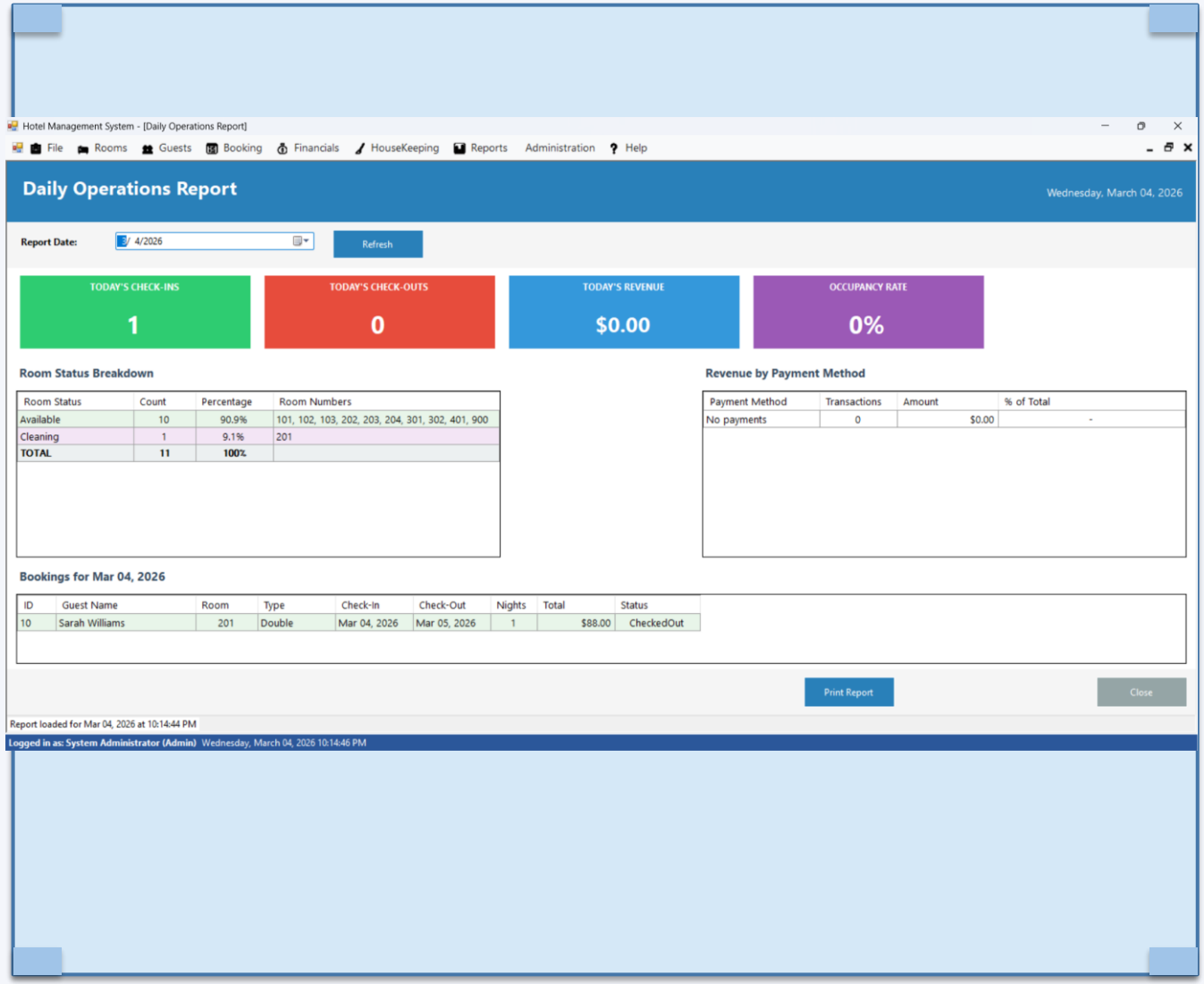
- Full CRUD operations for staff accounts
- Add / Edit via AddEditStaffDialog
- Role assignment: Admin · Receptionist · Housekeeping
- Password management with SHA-256 secure hashing
- Admin can reset any staff member's password
- Restricted access — visible only to Admin role

💡 Admin-only: role-based access control enforced

FORM 12

Form 12 — Daily Operations Report

DailyOperationsReportForm.cs



Form Description

📌 **Purpose** Dashboard providing a real-time overview of hotel operations

Key Features & Functions

- Daily operations summary at a glance
- Key metrics: occupancy rate, bookings, revenue
- Active bookings, pending check-ins, pending check-outs
- Outstanding invoices and payment summaries
- Housekeeping task completion status
- Management tool for informed decision-making

💡 Real-time management dashboard — all KPIs in one view

PATTERNS

Design Patterns — Overview

5 patterns implemented from textbook

Each pattern solves a real architectural challenge, demonstrating Separation of Concerns, Loose Coupling, and the Open/Closed Principle.

1 🔷 Singleton

DatabaseManager · RoomSubject

Guarantees only ONE instance of critical resources exists at all times.

2 🏭 Factory Method

RoomFactory · Room subclasses

Creates the correct room type object without exposing creation logic to UI.

3 ♟ Strategy

IPaymentStrategy · Cash · CreditCard

Swaps payment algorithms at runtime — add new methods without touching existing code.

4 👁 Observer

RoomSubject · HousekeepingObserver

Auto-triggers housekeeping task creation every time a guest checks out.

5 🎨 Decorator

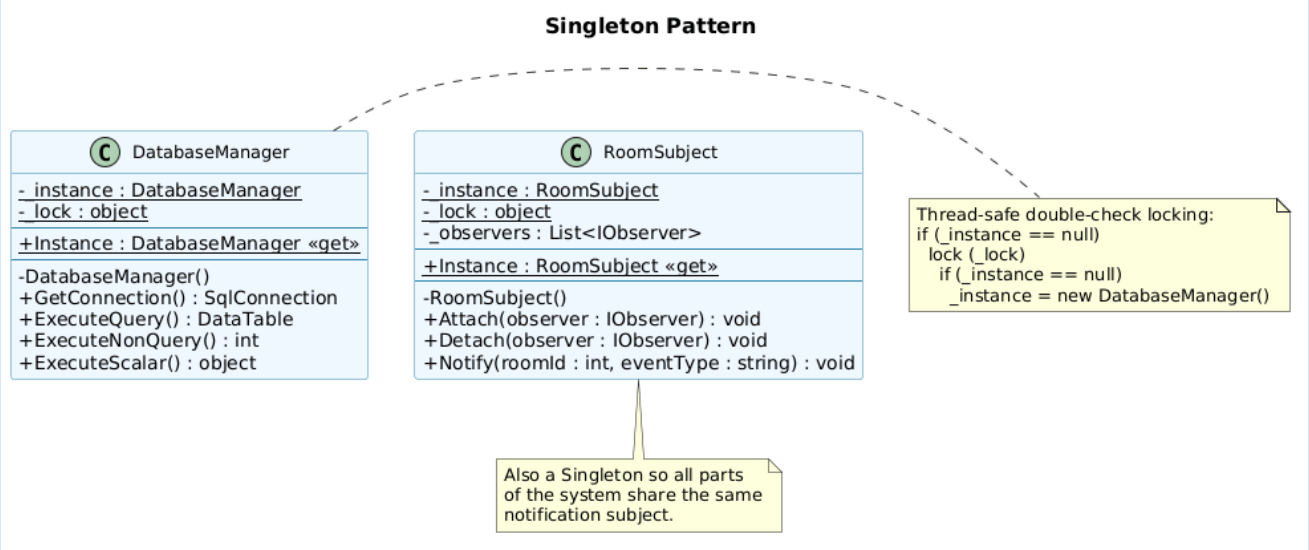
RoomDecorator · 3 Concrete Decorators

Dynamically adds room add-ons (breakfast, extra bed, transfer) without modifying Room.

PATTERN 1

Pattern 1 — Singleton

Files: DAL/DatabaseManager.cs · Patterns/RoomSubject.cs



Singleton Pattern

Purpose Ensures only ONE instance of a class exists throughout app lifetime

Where It's Used

- DatabaseManager — single shared database connection pool
- Thread-safe double-check locking (lock + null check x2)
- Private constructor prevents external instantiation
- RoomSubject — single event hub for all Observer notifications
- Every Repository uses DatabaseManager.Instance to query DB

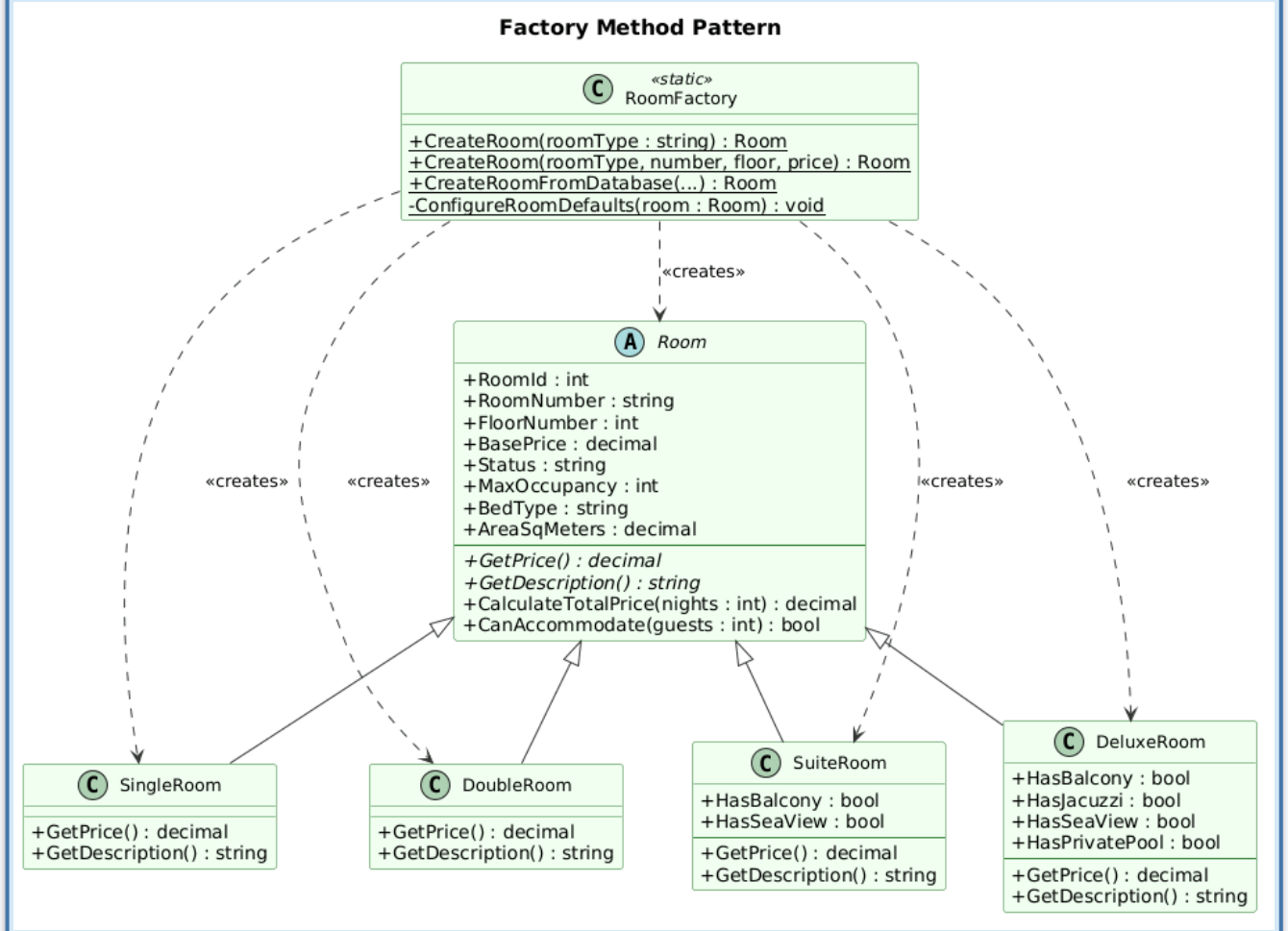
Why It Matters

- Prevents connection pool exhaustion from multiple instances
- Guarantees state consistency across multi-threaded UI events
- Centralizes DB connection management in one place

PATTERN 2

Pattern 2 — Factory Method

Files: BLL/Factories/RoomFactory.cs · Models/Room.cs + 4 subclasses



Factory Method Pattern

Purpose Encapsulates creation of different room types without exposing concrete classes

Where It's Used

- RoomFactory.CreateRoom(type) returns correct subclass
- Single → MaxOccupancy 1 · $50/night · Single bed · 20 m²
- Double → MaxOccupancy 2 · $80/night · Queen bed · 30 m²
- Suite → MaxOccupancy 4 · $150/night · King · Balcony
- Deluxe → MaxOccupancy 4 · $250/night · King · Jacuzzi
- CreateRoomFromDatabase() reconstructs rooms loaded from SQL

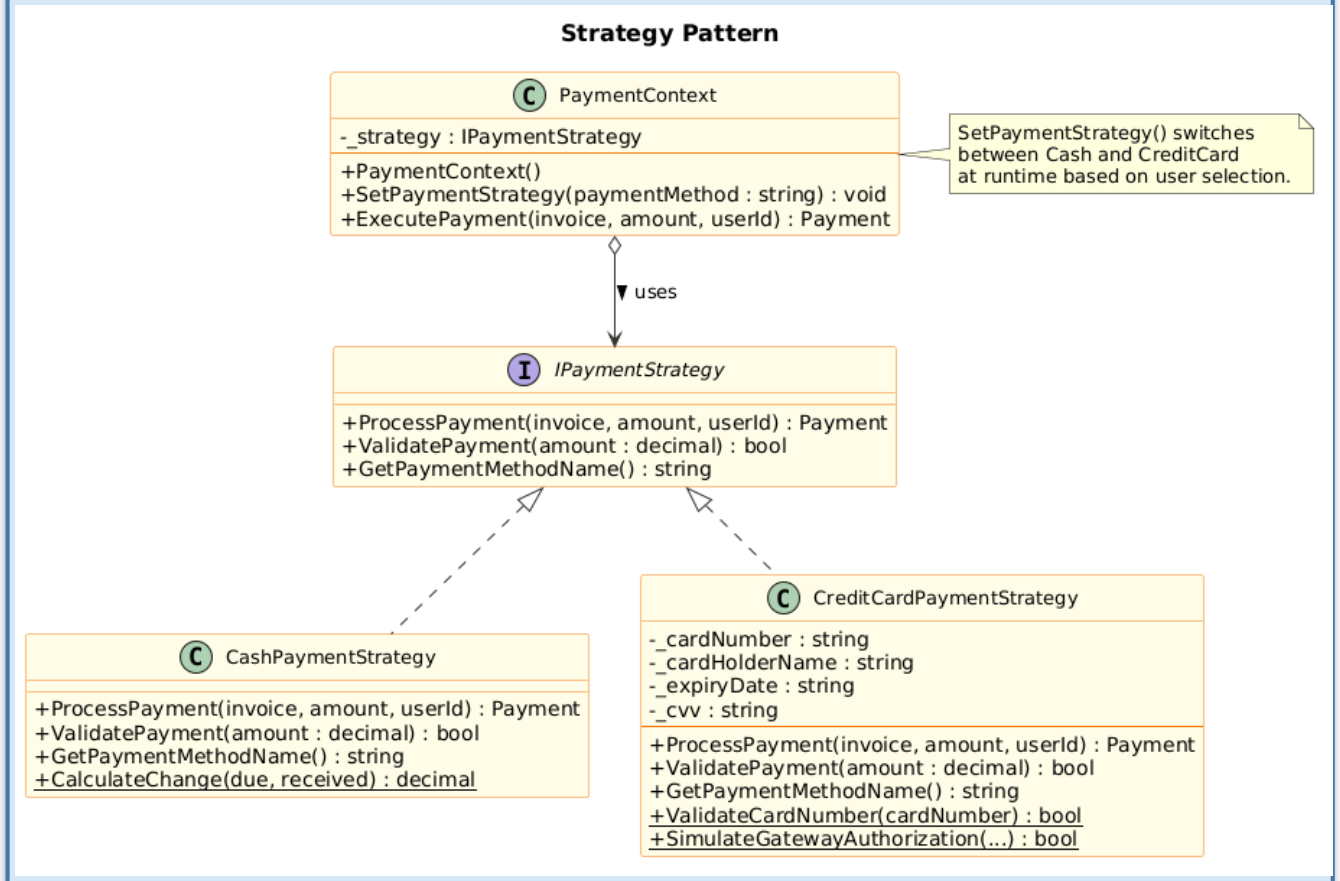
Why It Matters

- UI forms never reference concrete room classes directly
- All default configs (bed, area, price) defined in one location
- New room type = add one new subclass only, no UI changes

PATTERN 3

Pattern 3 — Strategy

Files: Patterns/IPaymentStrategy.cs · CashPaymentStrategy.cs · CreditCardPaymentStrategy.cs · PaymentContext.cs



Strategy Pattern

Purpose Defines interchangeable payment algorithms — selected at runtime

Where It's Used

- IPaymentStrategy interface: ProcessPayment() + Validate()
- CashPaymentStrategy → calculates change, creates Cash payment
- CreditCardPaymentStrategy → Luhn validation, CVV check, gateway simulation (95% success), masks card number
- PaymentContext.SetStrategy('Cash'/'CreditCard') at runtime
- PaymentForm selects strategy based on user's radio button

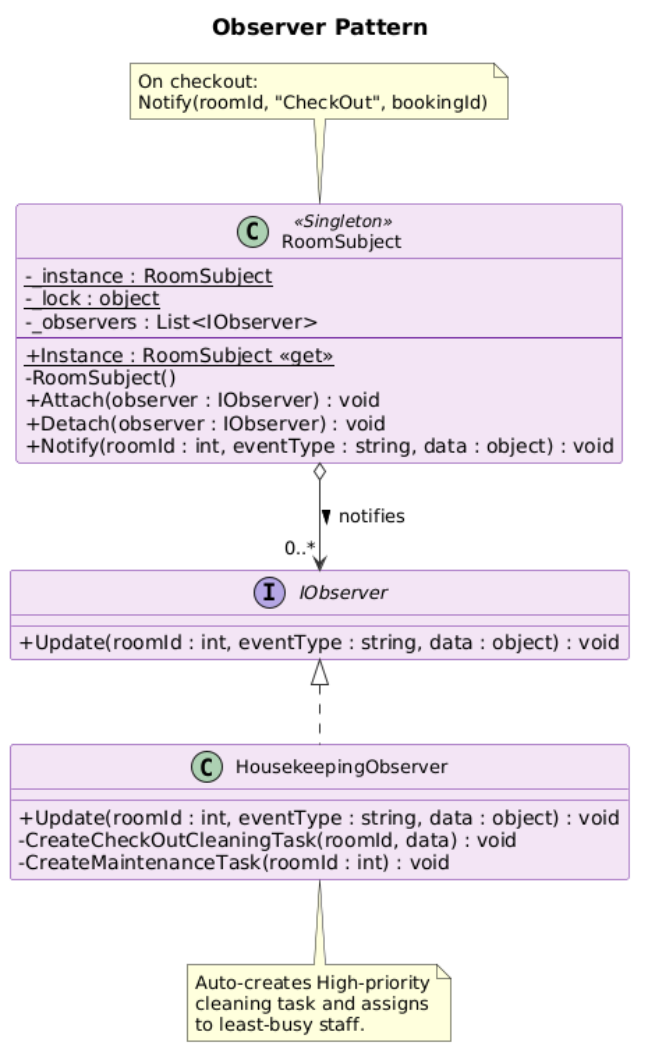
Why It Matters

- Adding Mobile Pay / Bank Transfer = one new class only
- Existing payment code is never modified (Open/Closed Principle)
- Each strategy is independently testable

PATTERN 4

Pattern 4 — Observer

Files: Patterns/IObserver.cs · Patterns/RoomSubject.cs · Patterns/HousekeepingObserver.cs



Observer Pattern

Purpose One-to-many event system — dependents notified automatically on state change

Where It's Used

- RoomSubject (Singleton) holds a list of registered IObservers
- MainForm.ctor → RoomSubject.Instance.Attach(new HousekeepingObserver())
- BookingListForm checkout → RoomSubject.Notify('CheckOut', roomId)
- HousekeepingObserver.Update() → auto-creates High-priority Cleaning Task
- Smart assignment: finds housekeeping staff with fewest active tasks
- Also handles 'Maintenance' event type for maintenance tasks

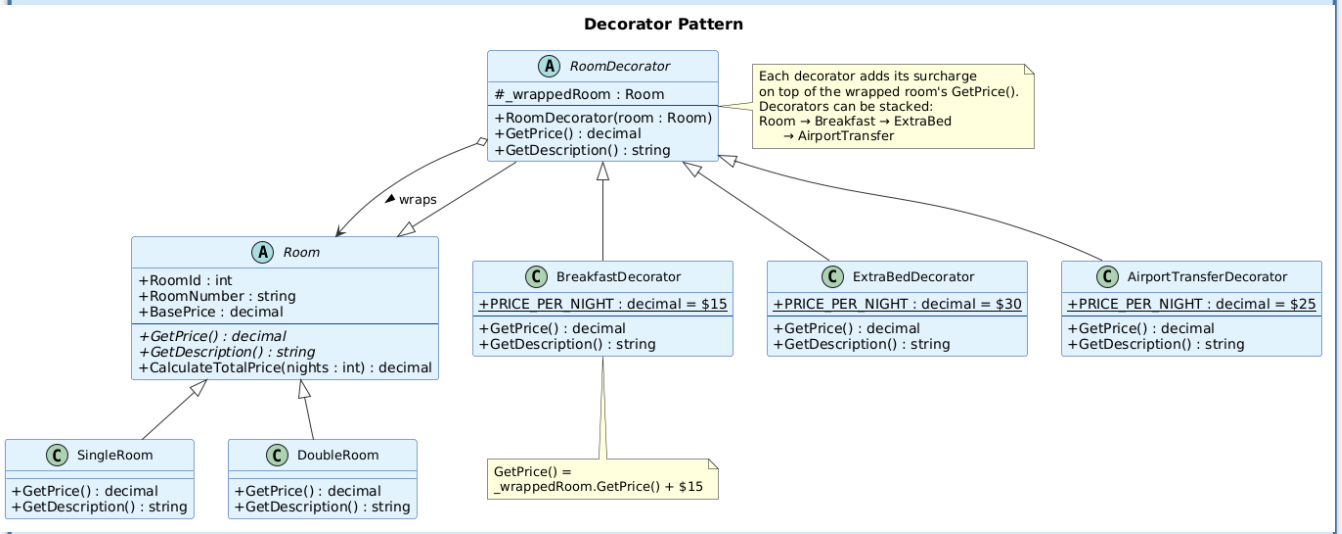
Why It Matters

- Booking module and Housekeeping module are fully decoupled
- New automated actions on checkout = add a new Observer only
- No changes needed to existing checkout or booking code

PATTERN 5

Pattern 5 — Decorator

Files: Patterns/RoomDecorator.cs · BreakfastDecorator.cs · ExtraBedDecorator.cs · AirportTransferDecorator.cs



Decorator Pattern

Purpose Attaches additional responsibilities to objects dynamically without subclassing

Where It's Used

- RoomDecorator — abstract base class wrapping any Room instance
- BreakfastDecorator → adds +$15/night breakfast service charge
- ExtraBedDecorator → adds +$30/night extra bed service charge
- AirportTransferDecorator → adds +$25/night airport transfer
- NewBookingForm stacks decorators from user-selected checkboxes
- decorated.GetPrice() - baseRoom.GetPrice() = total add-on charges

Why It Matters

- Add-ons applied at runtime — no Room subclass modifications needed
- Decorators stack: all three services combine transparently
- Open/Closed Principle: new add-on = one new class, zero changes elsewhere

CONCLUSION

Conclusion

Summary of the system's development and key achievements

Secure Authentication

SHA-256 hashing with random salt — user passwords are never stored in plain text.

Automated Workflows

Observer pattern: guest checkout auto-triggers housekeeping task creation with smart staff assignment.

Flexible Payments

Strategy pattern: new payment methods require only one new class — no existing code modified.

Clean Data Layer

Generic IRepository<T> with 7 implementations — consistent, parameterized, injection-safe SQL.

Simplified Booking

Facade pattern reduces an 8-step multi-repository workflow to a single validated method call.

Robust Validation

ValidationHelper covers email, phone, Luhn (credit card), date ranges, and numeric inputs.

The Hotel Management System delivers a fully functional platform covering the complete hotel lifecycle: guest registration → booking → check-in → stay → checkout → payment → housekeeping, underpinned by 6 design patterns and a clean 3-layer architecture.

System Architecture Summary

Presentation Layer : 15+ Windows Forms — Login, Booking, Payment, Receipt, Housekeeping, Reports …
Business Logic Layer : BookingFacade · RoomFactory · PasswordHelper · ValidationHelper
Design Patterns : Singleton · Factory Method · Strategy · Observer · Repository · Facade
Data Access Layer : DatabaseManager (Singleton) + 7 Repository implementations

DEMO

Demo — Live System Operations

Run the program and demonstrate actual CRUD and workflow operations

➕ Insert

Add a new guest
Add a room
Create a booking

✏️ Update

Edit guest profile
Modify room details
Confirm a booking

🔍 Search

Search guest by name
Filter rooms by status
Filter bookings

🗑 Delete

Soft-delete a guest
Cancel a booking
Remove staff account

🔐 Login

Login as Admin
Login as Receptionist
Compare role views

💳 Payment

Process cash
Process credit card
View styled receipt

🧹 Housekeep

Check out a guest
See auto-task created
Update task status

📊 Report

View daily dashboard
Check occupancy rate
Review invoices